Milestone 3: AI-Powered Test Case Generator for Software Testing

Class: "DSC670-T301 Advance Uses of Generative AI (2265-1)
Student: "Roshan GC"
Professor: "Frank Neugebauer"
Date: "05/10/2026"

Purpose

This notebook follows from previous Milestone-2 and it aims to build a fine-tuned OpenAI model for producing software test cases. An objective is to move from past prompt experiments by training a model on QA examples such that the results are more consistent with the practices of software testing. This notebook covers data set creation, uploading files, creating fine-tune jobs, endpoints tracking, metrics analysis, testing model, and commentary.

Import required libraries and setting up the OpenAI client

In [13]:
```
# importing required libraries
import os
import json
import time
from pathlib import Path
from datetime import datetime

import pandas as pd
from openai import OpenAI

client = OpenAI(api_key="sk-proj-NwQzwsAE3z9MMEI9DXT9gxuIG0eDjtGBZogjOR97qMr

BASE_MODEL = "gpt-3.5-turbo"

PROJECT_DIR = Path("AI-Powered-Test-Case-Generator")
PROJECT_DIR.mkdir(exist_ok=True)
```

Dataset

For the fine-tuning, I generated a small QA-oriented dataset. Each entry has a software requirement as input and structured test cases as output. The dataset trains the model on how to generate test case IDs, scenario descriptions, categories, steps etc.

In [28]:
```
# Defining the system message
system_message = (
    "You are a QA test case generation assistant. "
    "Given a software requirement, generate concise, structured test cases.
    "Use markdown table format with columns: TC ID, Scenario, Category, Step
)

# Sample dataset
examples = [
    {
        "requirement": "Login feature: username and password are required. V
        "answer": "| TC ID | Scenario | Category | Steps | Expected Result |
    },
    {
        "requirement": "Registration form: email, password, and confirm pass
        "answer": "| TC ID | Scenario | Category | Steps | Expected Result |
    },
    {
        "requirement": "Shopping cart: user can add products, update quantit
        "answer": "| TC ID | Scenario | Category | Steps | Expected Result |
    },
    {
        "requirement": "Password reset: user enters registered email, receiv
        "answer": "| TC ID | Scenario | Category | Steps | Expected Result |
    },
    {
        "requirement": "Search feature: user can search products by keyword.
        "answer": "| TC ID | Scenario | Category | Steps | Expected Result |
    },
    {
        "requirement": "Checkout: user must provide shipping address, paymer
        "answer": "| TC ID | Scenario | Category | Steps | Expected Result |
    },
    {
        "requirement": "Profile update: user can update name, phone number,
        "answer": "| TC ID | Scenario | Category | Steps | Expected Result |
    },
    {
        "requirement": "File upload: user can upload PDF files up to 5 MB. C
        "answer": "| TC ID | Scenario | Category | Steps | Expected Result |
    },
    {
        "requirement": "Order history: user can view previous orders, filter
        "answer": "| TC ID | Scenario | Category | Steps | Expected Result |
    },
    {
        "requirement": "Logout feature: user can log out from the applicatic
        "answer": "| TC ID | Scenario | Category | Steps | Expected Result |
    },
    {
        "requirement": "Admin user management: admin can create, edit, deact
        "answer": "| TC ID | Scenario | Category | Steps | Expected Result |
    },
    {
        "requirement": "Notification settings: user can enable or disable em
```

```
        "answer": "| TC ID | Scenario | Category | Steps | Expected Result |
    },
    {
        "requirement": "Product review: logged-in users can submit a rating
        "answer": "| TC ID | Scenario | Category | Steps | Expected Result |
    },
    {
        "requirement": "Coupon code: user can apply a valid coupon during ch
        "answer": "| TC ID | Scenario | Category | Steps | Expected Result |
    },
    {
        "requirement": "Appointment booking: user selects service, date, tim
        "answer": "| TC ID | Scenario | Category | Steps | Expected Result |
    }
]
```

Convert Dataset to JSONL Format

Since OpenAI fine-tuning requires JSONL data. Each line is a JSON object containing messages. I splitted the examples into a training file and a validation file.

In [29]:
```
# Each example is converted into a JSONL
def make_record(example):
    return {
        "messages": [
            {"role": "system", "content": system_message},
            {"role": "user", "content": example["requirement"]},
            {"role": "assistant", "content": example["answer"]}
        ]
    }

records = [make_record(ex) for ex in examples]

train_records = records[:12]
valid_records = records[12:]

train_path = PROJECT_DIR / "qa_testcase_train.jsonl"
valid_path = PROJECT_DIR / "qa_testcase_validation.jsonl"

def write_jsonl(path, rows):
    with open(path, "w", encoding="utf-8") as f:
        for row in rows:
            f.write(json.dumps(row) + "\n")

write_jsonl(train_path, train_records)
write_jsonl(valid_path, valid_records)

#printing dataset information
print("Training file:", train_path)
print("Validation file:", valid_path)
print("Training examples:", len(train_records))
print("Validation examples:", len(valid_records))
```

```
Training file: AI-Powered-Test-Case-Generator/qa_testcase_train.jsonl
Validation file: AI-Powered-Test-Case-Generator/qa_testcase_validation.jsonl
Training examples: 12
Validation examples: 3
```

Validation

I checked every line is valid JSON and contains system, user, and assistant messages.

In [30]:
```python
# Validation
def validate_jsonl(path):
    errors = []
    count = 0

    with open(path, "r", encoding="utf-8") as f:
        for line_number, line in enumerate(f, start=1):
            count += 1
            try:
                obj = json.loads(line)
            except json.JSONDecodeError as e:
                errors.append(f"Line {line_number}: JSON error - {e}")
                continue

            if "messages" not in obj:
                errors.append(f"Line {line_number}: missing messages")
                continue

            roles = [m.get("role") for m in obj["messages"]]
            if roles != ["system", "user", "assistant"]:
                errors.append(f"Line {line_number}: roles are {roles}, expec

    return count, errors

for path in [train_path, valid_path]:
    count, errors = validate_jsonl(path)
    print(f"{path.name}: {count} records")
    if errors:
        for err in errors:
            print("-", err)
    else:
        print("No validation errors.")
```

```
qa_testcase_train.jsonl: 12 records
No validation errors.
qa_testcase_validation.jsonl: 3 records
No validation errors.
```

Preview Training Example

This preview confirmed, the model will receive a requirement and learn the desired structured response format.

In [31]:
```
# Preview One Training Example
with open(train_path, "r", encoding="utf-8") as f:
    first_line = json.loads(f.readline())

print(json.dumps(first_line, indent=2))
```


```
{
  "messages": [
    {
      "role": "system",
      "content": "You are a QA test case generation assistant. Given a softw
are requirement, generate concise, structured test cases. Use markdown table
format with columns: TC ID, Scenario, Category, Steps, Expected Result."
    },
    {
      "role": "user",
      "content": "Login feature: username and password are required. Valid c
redentials allow login. Invalid credentials show an error. Account locks aft
er five failed attempts."
    },
    {
      "role": "assistant",
      "content": "| TC ID | Scenario | Category | Steps | Expected Result |
\n|---|---|---|---|---|\n| TC_LOGIN_001 | Login with valid credentials | Fun
ctional | Open login page; enter valid username; enter valid password; click
Login | User logs in successfully and is redirected to dashboard |\n| TC_LOG
IN_002 | Submit blank username and password | Negative | Open login page; le
ave username blank; leave password blank; click Login | Required field valid
ation messages are displayed |\n| TC_LOGIN_003 | Login with invalid password
| Negative | Enter valid username; enter invalid password; click Login | Inv
alid credentials error is displayed |\n| TC_LOGIN_004 | Lock account after f
ive failed attempts | Boundary | Attempt login five times with wrong passwor
d | Account is locked after the fifth failed attempt |\n| TC_LOGIN_005 | Log
in after account is locked | Negative | Use locked account credentials; clic
k Login | Login is denied and locked account message is displayed |"
    }
  ]
}
```


Upload Training and Validation Files

The files are uploaded with purpose="fine-tune". The returned file IDs were used to create the fine-tuning job.

In [32]:
```
## Upload Training and Validation Files
train_file = client.files.create(
    file=open(train_path, "rb"),
    purpose="fine-tune"
)

valid_file = client.files.create(
    file=open(valid_path, "rb"),
    purpose="fine-tune"
)
```

```
print("Training file ID:", train_file.id)
print("Validation file ID:", valid_file.id)
```

```
Training file ID: file-KQhRYTJRttbXAw82dnbMyA
Validation file ID: file-26sYmcrjdbx61cbXqadPcm
```

Creating the Fine-Tuning Job

The fine-tuning job is created based on the uploaded data sets and the base model is trained by OpenAI on the provided QA examples to enable the model to understand the required style of the generated tests.

In [ ]:
```
# Create fine-tuning job
CREATE_FINE_TUNE_JOB = True

if CREATE_FINE_TUNE_JOB:
    fine_tune_job = client.fine_tuning.jobs.create(
        model=BASE_MODEL,
        training_file=train_file.id,
        validation_file=valid_file.id,
        suffix="qa-testcase-generator"
    )
    job_id = fine_tune_job.id
    print("Fine-tuning job created.")
    print("Job ID:", job_id)
    print("Status:", fine_tune_job.status)
else:
    job_id = "ADD_JOB_ID_HERE"
    print("Fine-tuning job is not created in this run.")
```

```
Fine-tuning job created.
Job ID: ftjob-nciXUCZpM4ss7kUU9HiCtV77
Status: validating_files
```

Retrieve Fine-Tuning Job Status

This tracks the job status, including whether it is validating files, queued, succeeded, failed etc.

In [48]:
```
# Retrieve fine-tuning job status
job = client.fine_tuning.jobs.retrieve(job_id)

print("Job ID:", job.id)
print("Status:", job.status)
print("Base model:", job.model)
print("Fine-tuned model:", job.fine_tuned_model)
print("Training file:", job.training_file)
print("Validation file:", job.validation_file)
print("Trained tokens:", job.trained_tokens)
print("Created at:", job.created_at)
print("Finished at:", job.finished_at)
```

```
if job.error:
    print("Error:", job.error)
```

```
Job ID: ftjob-nciXUCZpM4ss7kUU9HiCtV77
Status: running
Base model: gpt-3.5-turbo-0125
Fine-tuned model: None
Training file: file-KQhRYTJRttbXAw82dnbMyA
Validation file: file-26sYmcrjdbx61cbXqadPcm
Trained tokens: None
Created at: 1778366257
Finished at: 1778367729
Error: Error(code=None, message=None, param=None)
```

Tracking Job Events

This endpoint lists fine-tuning events. These events document what was happened during model build process.

In [49]:
```
# Tracking Job Events
events = client.fine_tuning.jobs.list_events(
    fine_tuning_job_id=job_id,
    limit=20
)

for event in events.data:
    print(event.created_at, "-", event.level, "-", event.message)
```

```
1778367734 - info - Evaluating model against our usage policies
1778367734 - info - New fine-tuned model created
1778367734 - info - Checkpoint created at step 84
1778367734 - info - Checkpoint created at step 72
1778367717 - info - Step 96/96: training loss=0.05, validation loss=0.81, fu
ll validation loss=0.70
1778367711 - info - Step 95/96: training loss=0.04, validation loss=0.52
1778367705 - info - Step 94/96: training loss=0.03, validation loss=0.76
1778367701 - info - Step 93/96: training loss=0.03, validation loss=0.51
1778367697 - info - Step 92/96: training loss=0.09, validation loss=0.77
1778367690 - info - Step 91/96: training loss=0.02, validation loss=0.81
1778367686 - info - Step 90/96: training loss=0.04, validation loss=0.76
1778367680 - info - Step 89/96: training loss=0.04, validation loss=0.52
1778367676 - info - Step 88/96: training loss=0.06, validation loss=0.80
1778367672 - info - Step 87/96: training loss=0.01, validation loss=0.77
1778367668 - info - Step 86/96: training loss=0.02, validation loss=0.50
1778367662 - info - Step 85/96: training loss=0.02, validation loss=0.76
1778367658 - info - Step 84/96: training loss=0.04, validation loss=0.77, fu
ll validation loss=0.67
1778367650 - info - Step 83/96: training loss=0.05, validation loss=0.75
1778367646 - info - Step 82/96: training loss=0.07, validation loss=0.49
1778367639 - info - Step 81/96: training loss=0.03, validation loss=0.48
```

Retrieve Metrics and Result Files

Once the fine-tuning job is succeded then the output files can be accessed to view critical data on training like job status, tokens trained, losses etc.

In [50]:
```
# retrieve job status until completion
job = client.fine_tuning.jobs.retrieve(job_id)

print("Status:", job.status)
print("Fine-tuned model:", job.fine_tuned_model)
print("Result files:", job.result_files)

if job.result_files:
    for result_file_id in job.result_files:
        print("\nDownloading result file:", result_file_id)
        try:
            result_content = client.files.content(result_file_id)

            if hasattr(result_content, "text"):
                text = result_content.text
            elif hasattr(result_content, "read"):
                text = result_content.read().decode("utf-8")
            else:
                text = str(result_content)

            print(text[:3000])
            result_path = PROJECT_DIR / f"{result_file_id}_results.txt"
            result_path.write_text(text, encoding="utf-8")
            print("Saved results to:", result_path)
        except Exception as e:
            print("Could not download or parse result file.")
            print("Error:", e)
else:
    print("No result files available yet. Wait until the job succeeds.")
```

```
Status: succeeded
Fine-tuned model: ft:gpt-3.5-turbo-0125:personal:qa-testcase-generator:DdkyU
Z2Y
Result files: ['file-93VwHPjkrs66MgDrNMSTwi']

Downloading result file: file-93VwHPjkrs66MgDrNMSTwi
```


```
c3RlcCx0cmFpbl9sb3NzLHRyYWluX2FjY3VyYWN5LHZhbGlkX2xvc3MsdmFsaWRfbWVhbl90b2tl
bl9hY2N1cmFjeQoxLDEuMTUwNjEsMC43NDIyNywxLjMyNTIsMC43NzUxNQoyLDEuNTQzNjEsMC43
MzYyLDEuMjgzNzIsMC43ODc4OAozLDEuNTU4NjIsMC43NTkyNiwxLjQ2NjYyLDAuNzI1NjEKNCwx
LjIwMzQyLDAuNzg1NzEsMS4xNzEyOSwwLjc5Mzk0CjUsMS40NTMxMywwLjc3Nzc4LDEuMTQyNjQs
MC43ODEwNwo2LDEuNDIwNjQsMC43OTc0NywxLjI0NzcsMC43Mzc4CjcsMS4zNzgzOSwwLjcyNTYx
LDEuMTQ1MywwLjczNzgKOCwwLjk1NTQsMC44MTAzNCwwLjg1MDQ1LDAuODEwNjUKOSwwLjgzODI4
LDAuODE3NjUsMC43MDkyMSwwLjc3NTc2CjEwLDEuMTYxNjcsMC43NDg3MiwwLjYzNjg4LDAuNzg3
ODgKMTEsMC43ODA3MiwwLjgwNTI2LDAuODY3ODYsMC43NjIyCjEyLDAuNTgzODUsMC43OTA3LDAu
NjM1OTgsMC43OTg4MgoxMywwLjUyNjYsMC43OTY1MSwwLjUzNzk4LDAuODI0MjQKMTQsMC44MjY3
MiwwLjc1NjEsMC44MDkwNCwwLjc4MDQ5CjE1LDAuNjEyOTcsMC43OTM4MSwwLjYwMTMsMC43OTg4
MgoxNiwwLjU2OTM2LDAuODI3NTksMC41OTE3MSwwLjgwNDczCjE3LDAuNTkxNTYsMC44MzUyOSww
LjUwMDkyLDAuODQyNDIKMTgsMC44MDA5LDAuNzY0MSwwLjc2NTM1LDAuNzc0MzkKMTksMC41Njk2
NywwLjgyNzM4LDAuNDc4MjIsMC44NTQ1NQoyMCwwLjU4MDQxLDAuODQxNzcsMC43NDU1NiwwLjc4
MDQ5CjIxLDAuNjAwMzYsMC44Mzk1MSwwLjU0NDE5LDAuODQ2MTUKMjIsMC42MTcwOSwwLjg0NjYz
LDAuNTMzMjcsMC44NTIwNwoyMywwLjQ3NCwwLjg0NzM3LDAuNzE4NDYsMC43ODY1OQoyNCwwLjU4
OTIsMC44NTk2NSwwLjQ0MDYzLDAuODYwNjEKMjUsMC40MzkyNiwwLjg3MzQyLDAuNDQwNTMsMC44
NTQ1NQoyNiwwLjU0NTgyLDAuODU5NjUsMC41MDk1OCwwLjg2OTgyCjI3LDAuMjkyMSwwLjkwMTE2
LDAuNjgwMTksMC43OTI2OAoyOCwwLjM3OTI5LDAuODczNjgsMC40MjQ1NSwwLjg2MDYxCjI5LDAu
MzcwMjcsMC44OTQxMiwwLjQ4NjY2LDAuODYzOTEKMzAsMC40NjAyLDAuODUzNjYsMC42NTg3Nyww
Ljc5MjY4CjMxLDAuMzk1NjksMC45MDEyMywwLjQ3Njc4LDAuODU3OTkKMzIsMC41NjI4LDAuODE1
MzgsMC42Mzk4NSwwLjc4NjU5CjMzLDAuMzU0NTIsMC44NzYyOSwwLjM5NDU1LDAuODY2NjcKMzQs
MC4zMjAzNSwwLjg4NjksMC42MjAwMSwwLjgxNzA3CjM1LDAuMzk1MzIsMC44Nzg3OQozNiwwLjM4
ODQxLDAuNDYxMDYsMC44Njk4MgozOCwwLjMyNjc3LDAuODk2MzQsMC42MDI3OSwwLjgwNDg4CjM5
LDAuMTY3OTcsMC45MzAyMywwLjM3MzM3LDAuODY2NjcKNDAsMC4yMTM3OSwwLjkxNTc5LDAuNTk5
NjgsMC44MDQ4OAo0MSwwLjI0Mzk2LDAuOTE5NzUsMC40NjM0NCwwLjg2MzkxCjQyLDAuNDIyMDcs
MC44NzE3OSwwLjM2MzM4LDAuODc4NzkKNDMsMC4yMDQ1NCwwLjkyNDA1LDAuNTk3ODIsMC44MDQ4
OAo0NCwwLjI1NDUyLDAuOTE3NTMsMC4zNjc5MywwLjg4NDg1CjQ1LDAuMjAwOTQsMC45MzUyOSww
LjQ4NDA0LDAuODYzOTEKNDYsMC4zMTQ1MywwLjkwMDU4LDAuNjAxLDAuODA0ODgKNDcsMC4yNDcz
LDAuOTMxMDMsMC40OTM5OSwwLjg1MjA3CjQ4LDAuMjQyODMsMC45MjYzOCwwLjM2OTYyLDAuODg0
ODUKNDksMC4xNDU2OCwwLjk2MzQ2LDAuNTA4ODUsMC44ODI3OSwwLjg3MjcwLjg0ODUKNTAsMC4x
ODIwNywwLjkzMjkzLDAuNTE2MDQsMC44NTIwNwo1MywwLjA5MjgzLDAuOTY1OTgsMC45NTEwMDgs
ODg0ODUKNTcsMC4xMDM4OSwwLjk3NjQ3LDAuNTU5MTEsMC44NDYxNQo2MCwwLjEyMTA4LDAuOTc2
MTksMC42MjE2MiwwLjgwNDg4CjYxLDAuMjExNDYsMC45MjgyMSwwLjU3MDAxLDAuODQ2MTUKNjIs
MC4wODM5OCwwLjk4ODI0LDAuMzk2OTgsMC44ODQ4NQo2MywwLjEzNTU5LDAuOTU0MDIsMC42Mzcx
NCwwLjc5MjY4CjY0LDAuMTE2ODIsMC45NTg3
```


```
Saved results to: AI-Powered-Test-Case-Generator/file-93VwHPjkrs66MgDrNMSTwi
_results.txt
```

Fine-Tuning Evaluation

The fine-tuning job was successful with the use of the gpt-3.5-turbo-0125 model as the base. Metrics including training loss, validation loss, and token accuracy were obtained

during the training process.Result shows, there is an improvement during training, meaning that the model learned the desired QA test case generation pattern.

Testing the Fine-Tuned Model

After the fine-tuning process completed successfully, the model was tested using a new software requirement that was not part of the training dataset. The purpose of this test was to evaluate whether the model can develop structured and relevant QA test-cases that were unknown.

In [ ]:
```
# Testing the Fine-Tuned Model

job = client.fine_tuning.jobs.retrieve(job_id)

FINE_TUNED_MODEL = job.fine_tuned_model

test_requirement = """
Two-factor authentication feature:
After entering valid username & password, the user must enter a six-digit ve
Invalid code should show an error.
Expired code should be rejected.
After three invalid code attempts, verification should be blocked.
"""

response = client.chat.completions.create(
    model=FINE_TUNED_MODEL,
    messages=[
        {"role": "system", "content": system_message},
        {"role": "user", "content": test_requirement}
    ],
    temperature=0.2
)

print(response.choices[0].message.content)
```

```
| TC ID | Scenario | Category | Steps | Expected Result |
|---|---|---|---|---|
| TC_AUTH_001 | Verify valid two-factor authentication | Functional | Enter
valid credentials; enter valid code | Logged in successfully |
| TC_AUTH_002 | Submit blank verification code | Negative | Enter valid cred
entials; leave code blank; submit | Required code message is displayed |
| TC_AUTH_003 | Enter invalid verification code | Negative | Enter valid cre
dentials; enter invalid code; submit | Invalid code error is displayed |
| TC_AUTH_004 | Use expired verification code | Negative | Enter valid crede
ntials; enter expired code; submit | Code expired message is displayed |
| TC_AUTH_005 | Reach maximum invalid attempts | Boundary | Enter valid cred
entials; enter wrong code three times | Verification is blocked after third
attempt |
| TC_AUTH_006 | Enter code after block | Negative | Use blocked account cred
entials; enter valid code | Verification is rejected |
```

Evaluation of Outupt

The tuned model managed to produce well-structured test cases for a two-factor authentication feature which was not part of the training dataset. Test cases followed the required QA format and covered various aspects such as functionality, negatives, and boundaries. This means that the model understood the required format for generating test cases through fine tuning.

Conclusion

This milestone has been quite successful in demonstrating the method by which a tuned AI model is trained and tested for test case generation in QA tasks. The model was trained on a set of QA data and was capable of generating structured test cases for a new task that was not present in the training dataset.

The outputs have adhered to the expected format of QA test cases and included test scenarios from the perspectives of functionality, negativity, and boundaries, indicate the model has certainly learned the QA format through the fine-tuning process.

Also, this milestone has enabled hands-on experience in several important aspects, such as preparing datasets, formatting in JSONL format, uploading files to the server, launching fine-tuning jobs, tracking endpoints, getting metrics, and testing the model.